

ORACLE

Oracle Converged Database Support for JSON data model

Witold Swierzy

Oracle EMEA Converged Database Expert

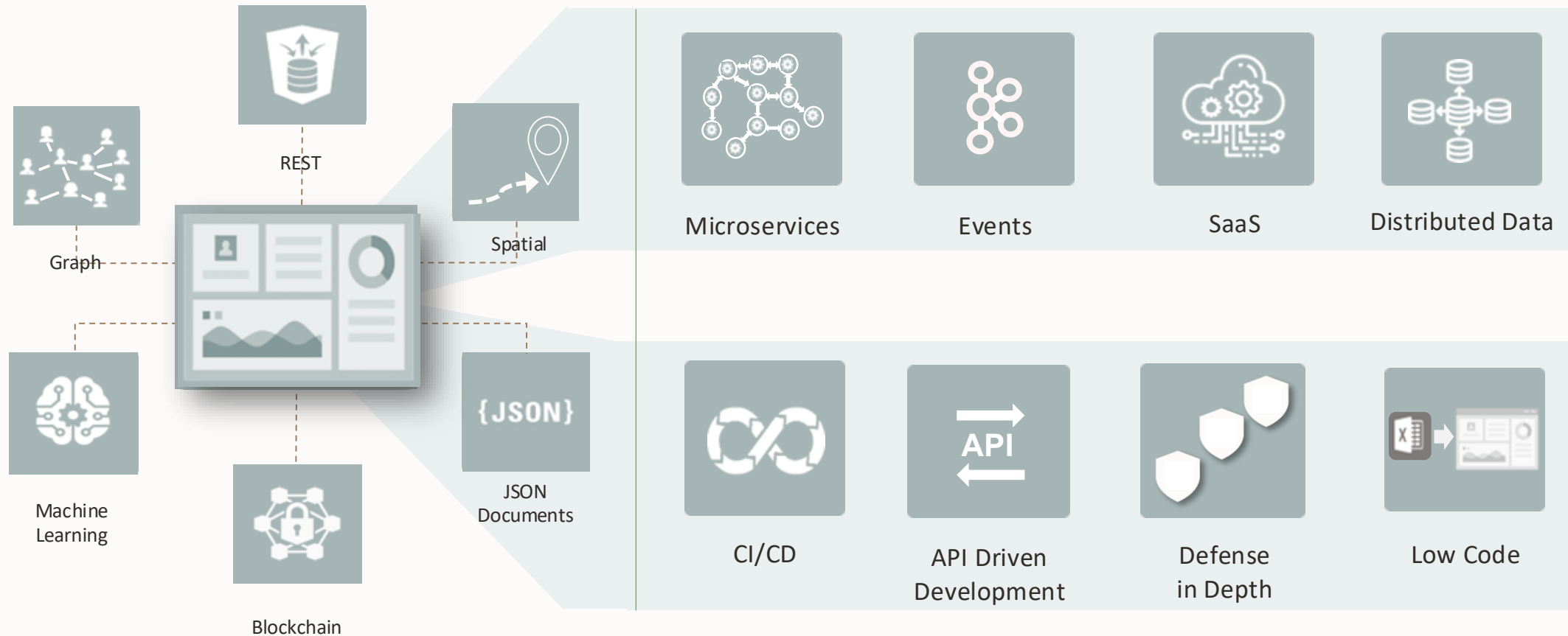


Introduction



Oracle Converged Database

Modern Apps Need To Generate Value From Data in New Ways They Are Built Using New Development Methodologies and Technologies



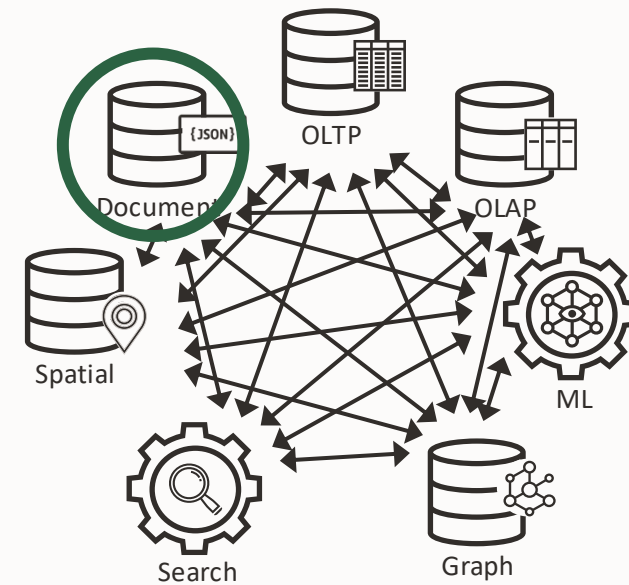
Oracle Converged Database

Converged Database Architecture



for **any** data type or workload
Developers focus on **Innovation**

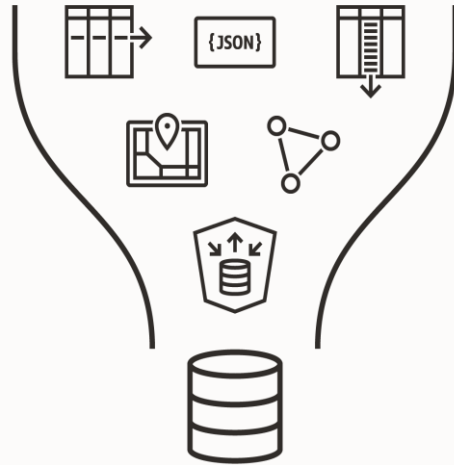
Single-purpose databases



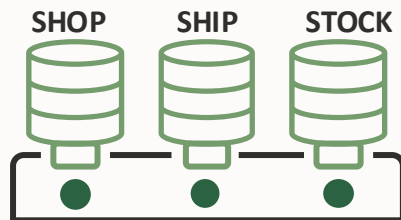
Multiple security models, languages, skills, licenses, etc
Developers focus on **Integration**

Oracle Database

A Converged Multi-Model, Multi-Workload Data Platform



Converged Database



Support for microservices

Multi-model

Best-of-Breed Relational, JSON, Spatial, Graph, Cube, Text, Blockchain
Cross-model operations enables you to easily create value across all your data

Multi-workload

High Performance Transactions, DW, Analytics, ML, IoT, Streaming
Deep optimizations deliver exceptional price-performance across all workloads

Most productive for developers and analysts

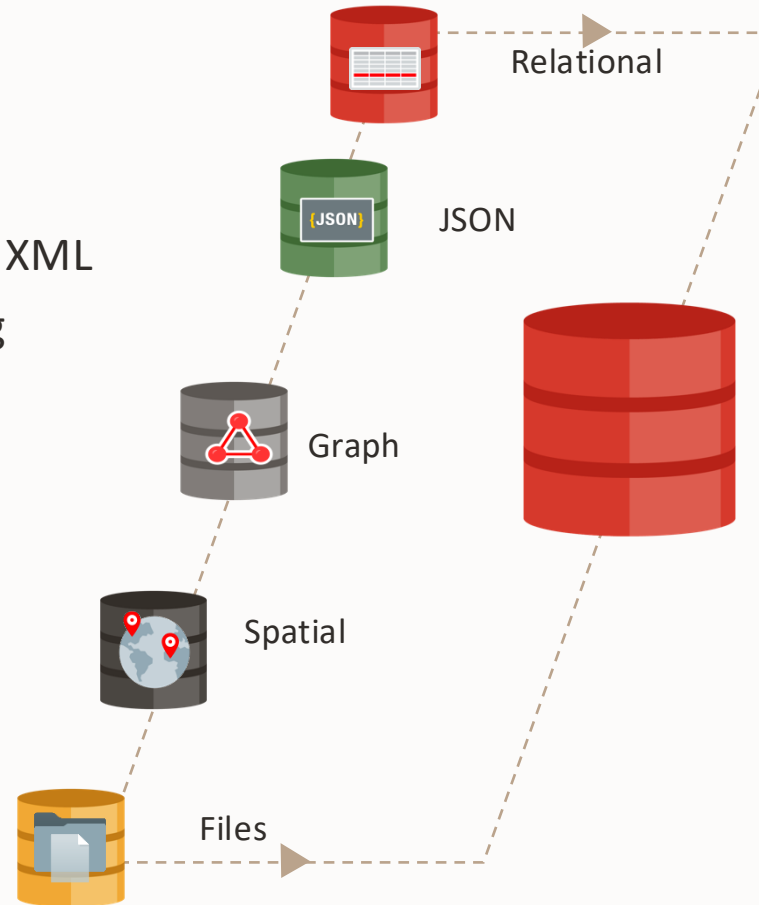
Same SQL and transactions operate on any data and workload
Integrated microservices, events, REST, CI/CD, Low-code

Support for microservices

Low-level data types and workloads should not dictate your architecture

Oracle Converged Cross-Model Database

- Any data, all workloads
- Full data synergy
- Best in class in **Every** Class:
 - Documents - JSON, Text, XML
 - AI and Machine Learning
 - Spatial and Graph
 - IoT, Times Series, Data, FS, Binary Data, etc.



Developers get what they want

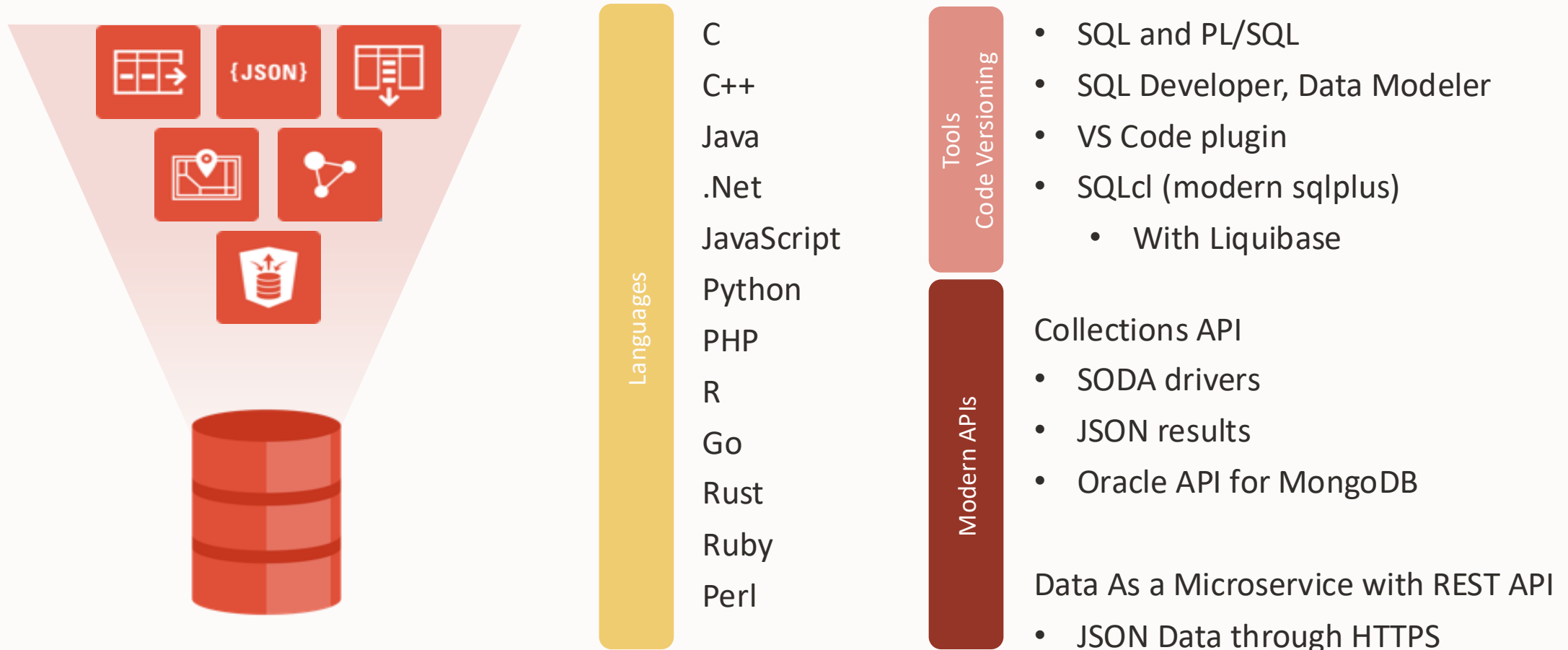
- Open Standards
- Simple APIs
- Polyglot
- Query across all data types
- Transactions across all data types

Analysts & Ops get what they need

- Consistent, queryable view of data
- Powerful query and analytics
- Reliability
- Scalability
- Security

Oracle Converged Database

support for all modern Languages/Drivers/Tools/APIs



Oracle Converged Database

JSON support

Relational vs Document data models

Comparison



Relational Data Model								
Database	Table	Parent-Child Tables	Row	Column	Index	Join	View	Multi-Record ACID Transaction
Database	Collection	Nested sub-document or array	Document	Field	Index	Embedding, Linkin, \$lookup	View	Multi-Document ACID Transaction
Document-Oriented Data Model								



Data Structures

JSON Collection Tables

- Data can be **stored** as **collections of JSON documents**
- Exposed as **collections** to MongoDB API

Storage Format

```
{
  name1 : String Value1,
  name2 :
    {
      name3 : 14:00,
      name4 : 1234
    }
}
```

ent Jill,

ita,

ce 102,

201,

SQL

```
CREATE JSON COLLECTION TABLE JOB_GRADES_JSON;
```

INSERT INTO EMPLOYEES VALUES

```
( '{"GradeLevel": "A", "LowestSal": 1000, ...}' );
```

MongoDB API

```
oradev> db.JOB_GRADES_JSON.find()
```

```
[ {
  _id: ObjectId('6a549305fb25c1dbb470394c'),
  GradeLevel: 'A',
  LowestSal: 1000,
  HighestSal: 2999
}
```

Data Structures

JSON Collection Views

- Read Only views
- Data can be **stored as rows in relational tables**
- Exposed as **collection views** to MongoDB API

Storage Format

TABLE		
Col 1	Col 2	Col 3
...	...	...
...	...	...
...	...	...
...	...	...

SQL

```
CREATE JSON COLLECTION VIEW EMP_VIEW  
AS  
SELECT JSON{*} FROM EMPLOYEES;
```

MongoDB API

```
db.EMP_VIEW.find()  
[  
  {  
    _id: ObjectId('69e9f97cc31199a063cbfc28' ),  
    EMPLOYEE_ID: 201,  
    LAST_NAME: 'Hartstein',  
    ...}  
  ...  
]
```

Data Structures

JSON-Relational Duality Views

- Read-Write views
- Data can be **stored as rows in relational tables**
- Exposed as **collection tables** to MongoDB API

Storage Format

TABLE		
Col 1	Col 2	Col 3
...	...	...
...	...	...
...	...	...
...	...	...

SQL

```
CREATE JSON RELATIONAL DUALITY VIEW DEPT_VIEW
AS
SELECT JSON{ '_id': DEPARTMENT_ID,
             'departmentName': DEPARTMENT_NAME,
             'locationId': LOCATION_ID }
FROM DEPARTMENTS
WITH INSERT UPDATE DELETE;
```

MongoDB API

```
oradev> db.DEPT_VIEW.insertOne(
{ '_id': 200, 'departmentName': 'Human Resources' })
{ acknowledged: true, insertedId: 200 }
```

Application Programming Interfaces

SQL

- Relational API, full support for all CRUD operations and ACID transactions
- Natively supported in all OracleDB-enabled applications and drivers

```
SQL> SELECT JSON_SERIALIZE(E.DATA PRETTY)
2  FROM EMP_VIEW E
3* WHERE E.DATA.SALARY > 17000;
```

```
JSON_SERIALIZE(E.DATA PRETTY)
```

```
-----
{
  "EMPLOYEE_ID" : 100,
  "EMAIL" : "SKING",
  "LAST_NAME" : "King",
  "SALARY": 24000
  ...
}
```

```
SQL> SELECT E.DATA.LAST_NAME, E.DATA.SALARY
2  FROM EMP_VIEW E
3* WHERE E.DATA.SALARY > 17000;
```

LAST_NAME	SALARY
"King"	24000

Application Programming Interfaces

Simple Oracle Document Access (SODA)

- NoSQL-style API, full support for all CRUD operations and ACID transactions
- Supported by Oracle tools, like SQLcl
- SODA Drivers for Java, C, REST, Node.JS, Python

```
SQL> SODA list
...
EMP_VIEW
DEPT_VIEW
...
SQL> SODA DROP DEPT_VIEW
Successfully dropped: DEPT_VIEW
```

```
SQL> SELECT E.DATA.LAST_NAME, E.DATA.SALARY
2  FROM EMP_VIEW E
3* WHERE E.DATA.SALARY > 17000;
```

LAST_NAME	SALARY
"King"	24000

Application Programming Interfaces

Oracle API for MongoDB



MongoDB Database compatibility layer

- Develop and run MongoDB workloads in the Oracle Database
- Full, unlimited ACID transaction support
- Supported by all Oracle Database options and features, for example
 - High Availability/Disaster Recovery (RAC, DataGuard, ...)
 - Security (Transparent Data Encryption, Data Masking and Subsetting, Oracle Label,...)
 - Analytics, ML/AI...
- Support for MongoDB native applications
 - Compatibility Level 7.x
 - Java, Node.JS, JavaScript,...
 - MongoDB tools (mongosh, mongodump/mongorestore, MongoDB Compass, ...)
- Available for all Oracle Database editions in all deployment models
 - No separate license required

Oracle Converged Database

—
Oracle API for MongoDB

Oracle API for MongoDB Architecture



- MongoDB application uses Mongo Wire Protocol to communicate with ORDS component
- ORDS acts as MongoDB compatibility layer
- ORDS translates MongoDB commands into sequences of SQL and PL/SQL calls
- ORDS communicates with Oracle Database using standard JDBC connection

Oracle API for MongoDB Architecture



Oracle Database Structure	Exposed to MongoDB applications as...
Schema	Database
JSON Collection Table JSON Relational Duality View	Collection
JSON Collection View	Collection View
Index	Index
Database User	Database User
Database Privilege	No 1:1 relationship between MongoDB and Oracle database privileges

Sample command running mongosh and connecting to an instance of Oracle API for MongoDB:

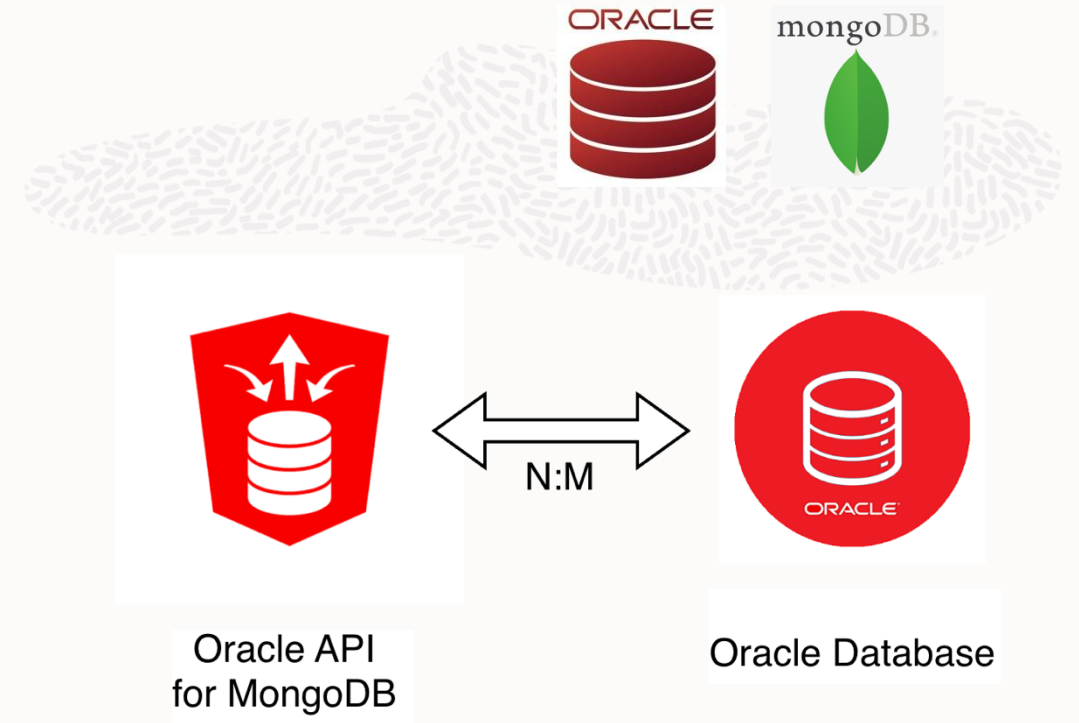
```
$ mongosh --tls 'mongodb://username:password@ORDS_machine_name:27017/database?...
```



Oracle API for MongoDB Architecture

- Many-to-many relationship between ORDS instances and Oracle Databases
- One ORDS instance can connect to multiple Oracle Database
- One Oracle Database can handle workload/connections coming from multiple ORDS instances
- Pluggable Databases Isolation

a single pluggable database is considered as a separate database



Oracle API for MongoDB

Installation and configuration



Oracle AI Autonomous Database case

- No need to install manually
- Not enabled by default
- Scaled automatically with Autonomous Database instance
- It is possible, but **not recommended** to configure an external ORDS instance

Details: [here](#)

The screenshot shows the Oracle Cloud console interface. At the top, there's a navigation bar with tabs: "Autonomous AI Database information", "Tool configuration" (which is selected), "Backups", "Key history", "Disaster recovery", "Refreshable clones", "Security", and "Monitoring". Below the navigation bar, the "MongoDB API" section is visible. It contains a description: "Oracle AI Database API for MongoDB enables MongoDB-compatible clients and drivers to connect directly to Autonomous AI Database. [Learn more](#)". Below this, there's a table with two rows. The first row is for the "Public access URL", showing a long MongoDB connection string and a "Copy" button. The second row is for the "Status", showing "Enabled" and an "Edit" button.

Autonomous AI Database information	Tool configuration	Backups	Key history	Disaster recovery	Refreshable clones	Security	Monitoring	>
------------------------------------	--------------------	---------	-------------	-------------------	--------------------	----------	------------	---

MongoDB API

Oracle AI Database API for MongoDB enables MongoDB-compatible clients and drivers to connect directly to Autonomous AI Database. [Learn more](#)

Public access URL	mongodb://[user:password@]G5E60EBADF0D95A-WSATP02.adb.eu-frankfurt-1.oraclecloudapps.com:27017/[user]?authMechanism=PLAIN&authSource=\$external&ssl=true&retryWrites=false&loadBalanced=true	Copy
Status	Enabled	Edit



Oracle API for MongoDB

Installation and configuration



General ORDS version guidelines

- Oracle recommends using the newest possible version of ORDS
- But.. what if we have a production system, which has been tested on an older ORDS version **only** ?

- This is  s  /  lemma

- Newest **(but not unstable)** component version with new features vs tested, stable **configuration**
- For production systems: **TEST IT FIRST. ALWAYS.**
- For dev/non-prod systems/systems being migrated from an original MongoDB database: **USE THE NEWEST AVAILABLE VERSION**

Oracle API for MongoDB

Installation and configuration



General Database version guidelines

- Oracle recommends using **26ai Oracle Database version**
- JSON support extended significantly
 - JSON storage (JSON Collection Tables, JSON Duality Views etc..)
 - JSON indexing (multivalue indexes, etc...)
- Some of important Oracle API for MongoDB functionalities require 26ai database version
 - vector processing, hybrid vector search, etc...

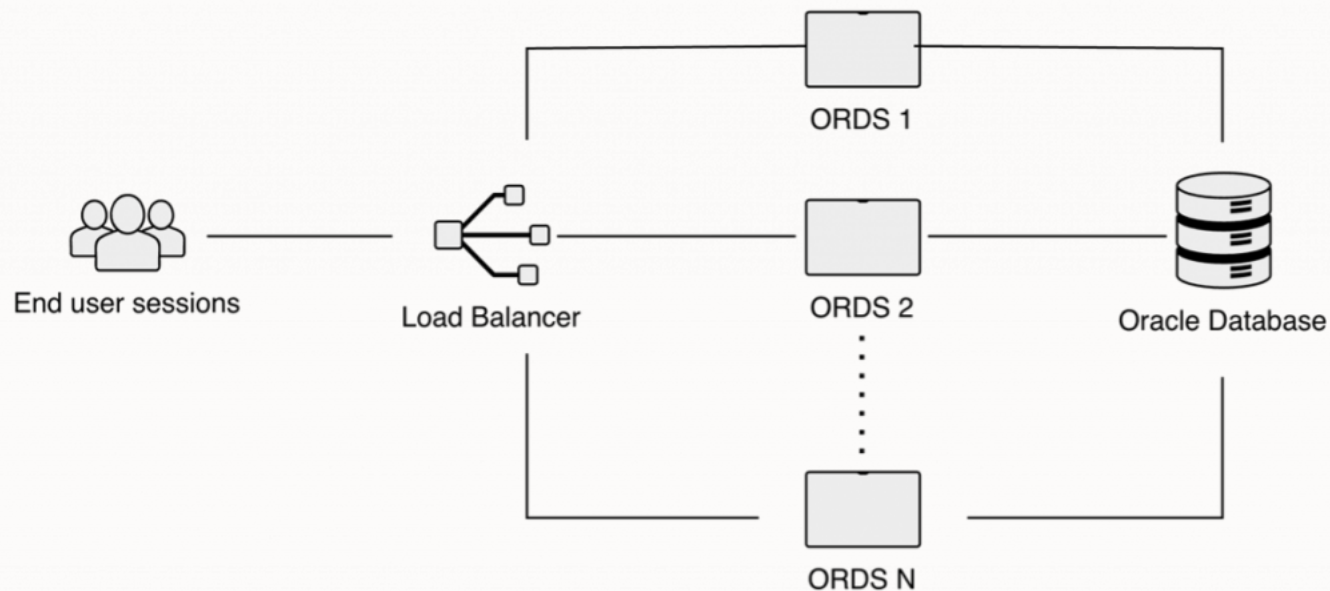
Oracle API for MongoDB

Scalability and High Availability



Important fact: MongoDB Wired Protocol is stateless

- Every request contains complete session and transaction information
- It is possible to use application-level load balancers as well as network-level load balancers with multiple ORDS instances as backend connected to the same database
- High Availability requirements may require using multiple instances of load balancer



Oracle API for MongoDB

Scalability and High Availability



Case #1: on-premises architecture

- HA Proxy is one of the most frequently used application-level load balancers
- Enterprise Edition provides High Availability features based on Virtual IP and multiple HA Proxy EE instances
- To build a HA architecture using **Community Edition** there is need to use additional solutions providing failover/switchover functionality
- Of course such architectures are not in any conflict with by Oracle Database options, including HA/DR solutions
 - RAC, (Active) DataGuard, Globally Distributed Database
- More details about configuring HA Proxy as load balancer for Oracle API for MongoDB can be found [here](#)
- **In fact it is possible to use ANY load balancer in exactly the same way**
- **Oracle DOES NOT provide suport for any third party components**

Oracle API for MongoDB

Scalability and High Availability



Case #2: OCI, non-autonomous Oracle Database

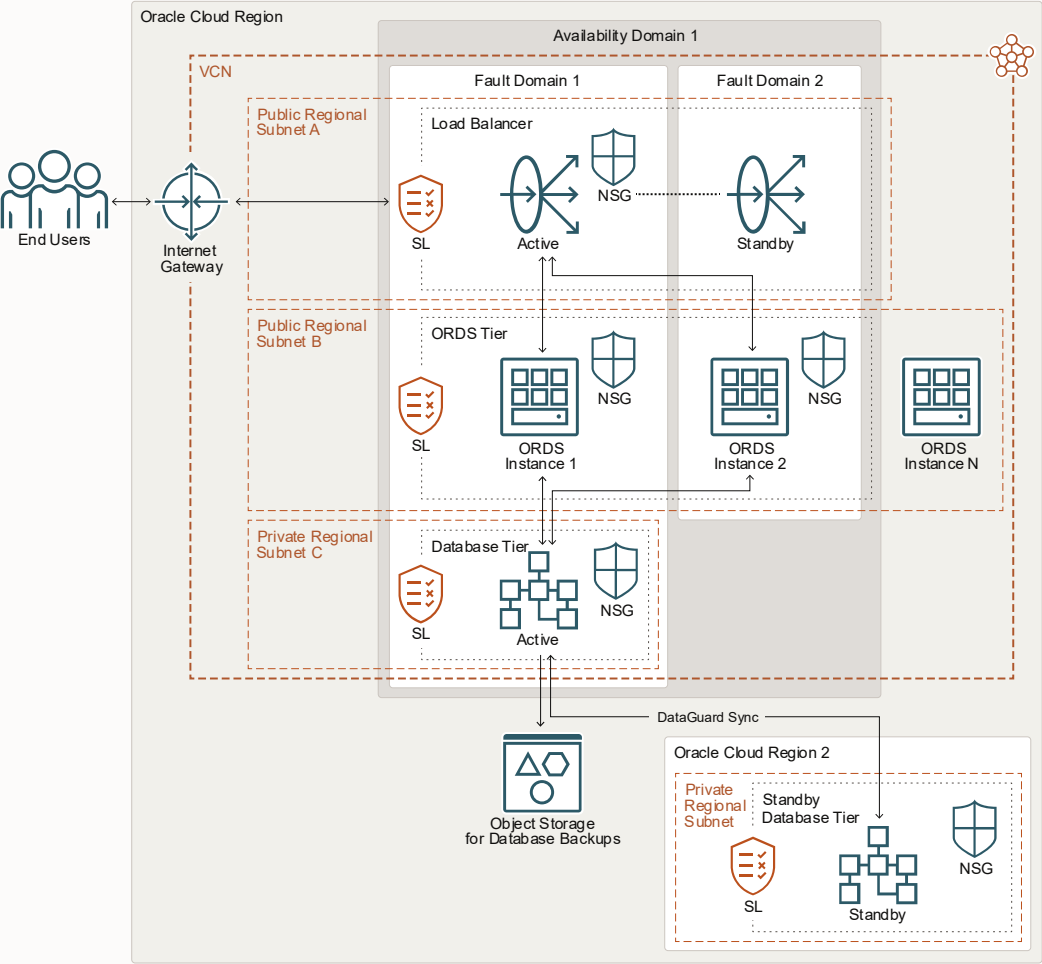
- In that case using OCI load balancers is the recommended way of building highly scalable and highly available systems
 - no manual configuration and excellent performance
- OCI provides both types of load balancers: application-level and network-level
- Application-level OCI Load Balancer (called officially simply Load Balancer) provides additional features, like DPI or throughput quotas
- Network-level OCI Load Balancer provides ability to configure backup backend-sets, which may be very important for HA architectures
- Use OCI DNS for configuring automatic failover
- Use (Active) DataGuard to HA/DR functionality at the database level

Oracle API for MongoDB

Scalability and High Availability



Case #2: OCI, non-autonomous Oracle Database



Oracle API for MongoDB

Scalability and High Availability



Case #3: OCI, Autonomous AI Oracle Database

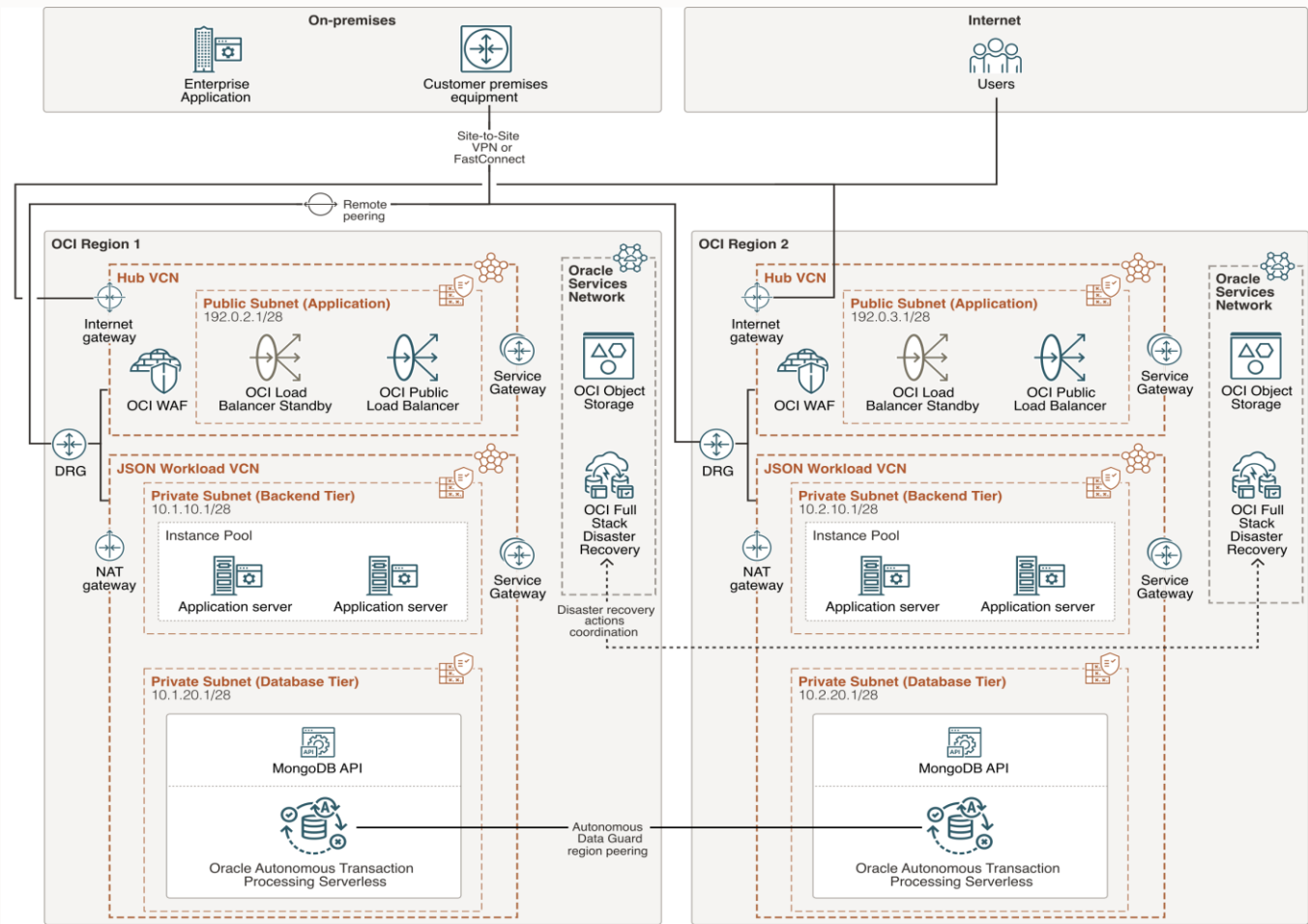
- ORDS built-in a service is scaled and maintained automatically
- In most cases there is no need to use a user-manager, external ORDS installations
- Use Oracle Autonomous DataGuard to provide HA/DR at the database level
 - Option 1, Autonomous AI Autonomous Transaction Processing with Autonomous Data Guard for disaster recovery
 - Option 2, Autonomous AI Autonomous Transaction Processing with Backup-Based Disaster Recovery.
- Use OCI Load Balancers for High Availability and Scalability

Oracle API for MongoDB

Scalability and High Availability



Case #3: OCI, Autonomous AI Oracle Database



Oracle API for MongoDB

Scalability and High Availability



Additional Resources

- [Oracle REST Data Services \(ORDS\) best practices \(official Oracle documentation\)](#)
- [Oracle Database API for MongoDB – Best Practices \(blog post\)](#)
- [Oracle API for MongoDB: Building and Scaling On-Premises Deployments \(blog post\)](#)

Oracle API for MongoDB

Deployment considerations



Deployment needs to be aligned with the existing infrastructure.

General recommendations

- **In most non-autonomous cases a standalone ORDS installation on a separate machine (virtual or bare metal) should be considered**
 - Maintenance
 - Security
 - Performance
- **In case, when an Autonomous Database service is used, ORDS integrated with the service should be used**
 - Automatic maintenance
 - Performance
 - No security violations

Oracle API for MongoDB

Deployment considerations



Key factors

- Security
- HA/DR
- Compatibility
- Performance

Complexity of infrastructure

- Multiple applications
- Multiple databases
- Isolated environments:
DEV,QA,UAT,PROD,...

Additional ORDS services

- APEX
- Database Actions
- REST

Oracle API for MongoDB

Deployment considerations



Key factors

Consider to isolate environments with different security requirements

- to avoid sensitive data thread

Consider to isolate environments with different HA/DR requirements

- Isolate critical environments from non-critical one

Consider to use separate environments for application with different compatibility requirements

- To avoid software conflicts

Performance impact

- Batch processing vs transaction processing
- Analytical processing vs transaction processing

Oracle API for MongoDB

Deployment considerations



Additional ORDS services

APEX vs Oracle API for MongoDB

- Security
- Performance impact caused by large APEX applications used by many users

REST vs Oracle API for MongoDB

- Security

Database Actions

- On demand queries
- On demand data load
- Security

Oracle API for MongoDB

Deployment considerations



DEV/QA/UAT/PROD environments isolation

- **Protect production data against end-user errors**
- **Protect sensitive data against unneeded access**
- **Performance protection**
- **Different HA/DR requirements**
- **Different lifecycle and maintenance requirements**
 - Patching
 - Upgrades
- **Different version requirements**
- **Different sizing**
 - DEV/QA vs UAT/PROD

Oracle API for MongoDB

Deployment considerations

Simple Example



Application	Stage	HA/DR requirements	Database requirement	version	ORDS version requirement	Data Sensitivity
App01	PROD	CRITICAL	23.26.1		25.4	LOW
App02	PROD	CRITICAL	23.26.1		25.4	LOW
App03	PROD	CRITICAL	23.26.1		25.4	HIGH
App04	DEV	LOW	23.26.2		26.2	LOW
App05	QA	LOW	23.26.1		25.4	LOW



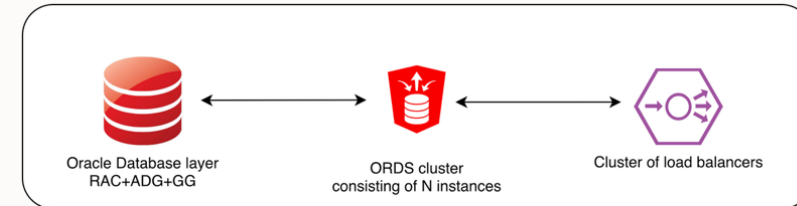
Oracle API for MongoDB

Deployment considerations

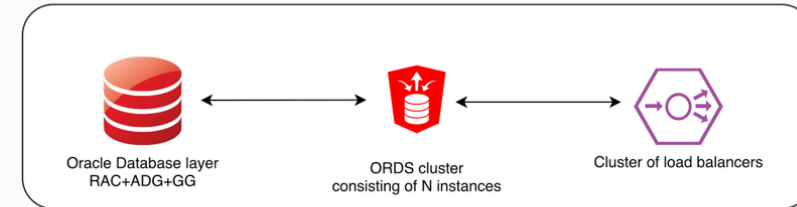
Simple Example

Result

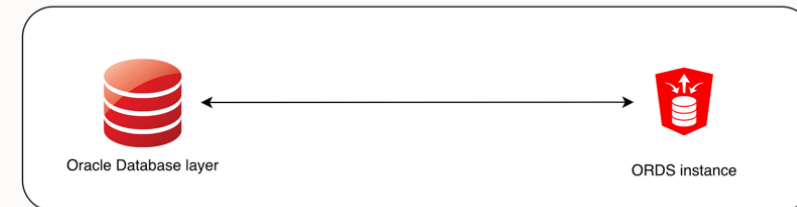
- Applications App01 and App02 can share ORDS and Database. Because of HA/DR and scalability requirements we should use a load balancer(s) in front of ORDS.
- Application App03 should be isolated because of data protection requirements. Because of HA/DR and scalability requirements we should use a load balancer(s) in front of ORDS.
- Applications App04 and App05 don't work on production data. They should use separate ORDS and database instances. Also they require different versions of ORDS and Database, so they cannot share the configuration. As they don't require any advanced HR/DR solutions, we don't need to configure load balancers for them.



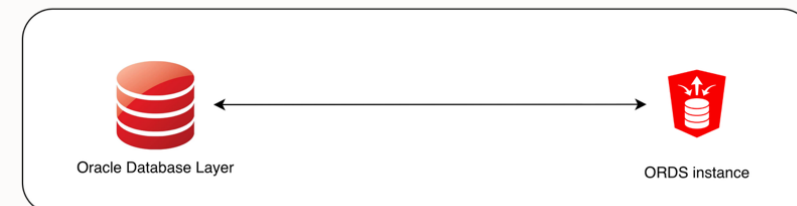
App01, App02



App03



App04



App05

Oracle API for MongoDB

ORDS Performance Tuning



- ORDS component DOES NOT consume significant amount of resources
- It does not perform any disk IO operations
- Example: An ORDS instance handling mongorestore proces of loading 20GB volume of the data with DOP=8

```
top - 06:08:57 up 6 min, 1 user, load average: 1.82, 0.77, 0.32
Tasks: 509 total, 1 running, 508 sleeping, 0 stopped, 0 zombie
%Cpu(s): 3.1 us, 0.2 sy, 0.0 ni, 96.6 id, 0.0 wa, 0.0 hi, 0.1 si, 0.0 st
MiB Mem : 15513.8 total, 11672.1 free, 2804.1 used, 1349.6 buff/cache
MiB Swap: 6144.0 total, 6144.0 free, 0.0 used. 12709.6 avail Mem

  PID USER      PR  NI   VIRT   RES    SHR S  %CPU  %MEM    TIME+  COMMAND
 5839 oracle    20   0   13.4g 971224 27960 S 102.0   6.1   3:40.03 java
 2103 root       20   0 3950624 547508 216756 S   0.3   3.4   0:17.85 falcon-sensor-b
    1 root       20   0   175360  18212  11356 S   0.0   0.1   0:01.86 systemd
    2 root       20   0         0         0         0 S   0.0   0.0   0:00.00 kthreadd
    3 root       20   0         0         0         0 S   0.0   0.0   0:00.00 pool_workqueue_release
    4 root        0 -20         0         0         0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
    5 root        0 -20         0         0         0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
    6 root        0 -20         0         0         0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
```



Oracle API for MongoDB

ORDS Performance Tuning



- Factors impacting the ORDS performance
 - network throughput and latency
 - two important tuneables/parameters managing the connection pool

Parameter	Description	Default Value
jdbc.MaxLimit	Maximum size of a JDBC connection pool	10
jdbc.InitialLimit	Initial number of JDBC connections in a pool	10



Oracle API for MongoDB

ORDS Performance Tuning



- `jdbc.MaxLimit`
maximum size of the connection pool
- `Jdbc.InitialLimit`
initial size of the connection pool
- Good starting point
 - set the `jdbc.MaxLimit` to the estimated maximum number of parallel sessions
 - set the `jdbc.InitialLimit` to the estimated average number of parallel sessions

Oracle API for MongoDB Migration



Offline Migration

- MongoDB native tools:
 - mongodump/mongorestore for multiple collections
 - mongoexport/mongoimport for a single collection
 - mongoexport/external table for a single collection

Online Migration

- Oracle GoldenGate for MongoDB migrations

Offline Migration to OCI

- Export data into OCI File Storage NFS volume
- Mount NFS volume in an OCI compute instance
- Use appropriate tools to import the data

Oracle API for MongoDB Migration



Performance considerations

- mongorestore tool consumes a lot of IO and CPU resources
 - use multiple CPU machine
 - use fast disks for a on-premises migration
 - set mount target throughput for a OCI NFS share to maximum value for a migration to OCI
- Network latency and throughput is another key factor, which needs to be considered
 - use private endpoints for a migration to OCI

their throughput is scaled automatically with an OCI compute instance and is not limited to 1Gb

Oracle API for MongoDB Migration



Compatibility considerations

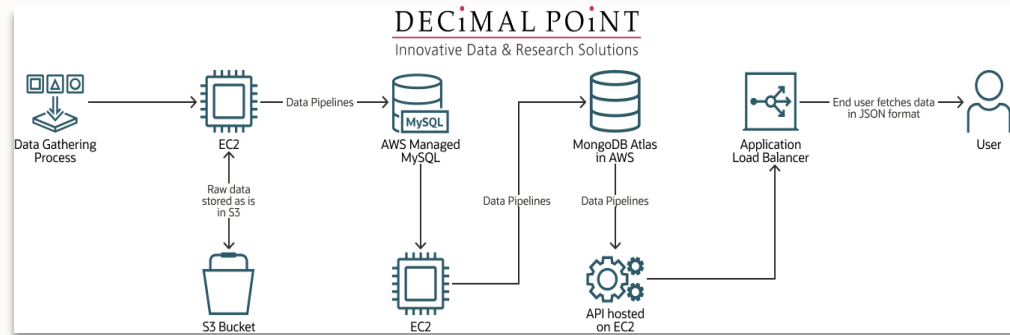
- Oracle API for MongoDB offers very high level of compatibility with the original DBMS
- Full support for CRUD operations
- High level of compatibility for aggregations
- Change Streams partial support
- Hybrid Vector Search partial support
- Oracle Estate Explorer for MongoDB tool can be used to
 - asses the compatibility level of the source system
 - discover potential problems as soon as possible, before the migration
- **Oracle AI Database 26ai and the latest available ORDS versions are strongly recommended**

Use Cases

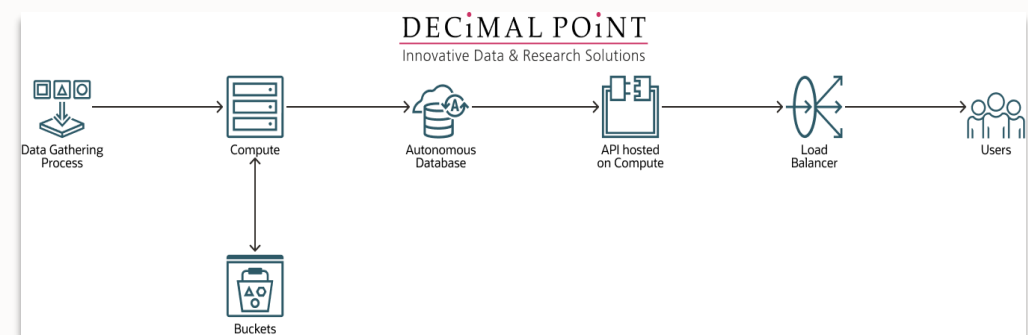
Converged Database – Success Story Decimal Point Analytics*

Migration from AWS [MySQL, MongoDB] to Oracle OCI [Autonomous Database]

AWS



OCI



Complex multi-database architecture on AWS

- Relational consistency through MySQL
- Data replication to MongoDB
- Complex, error-prone, and slow time-to-prod

New architecture using Autonomous Database

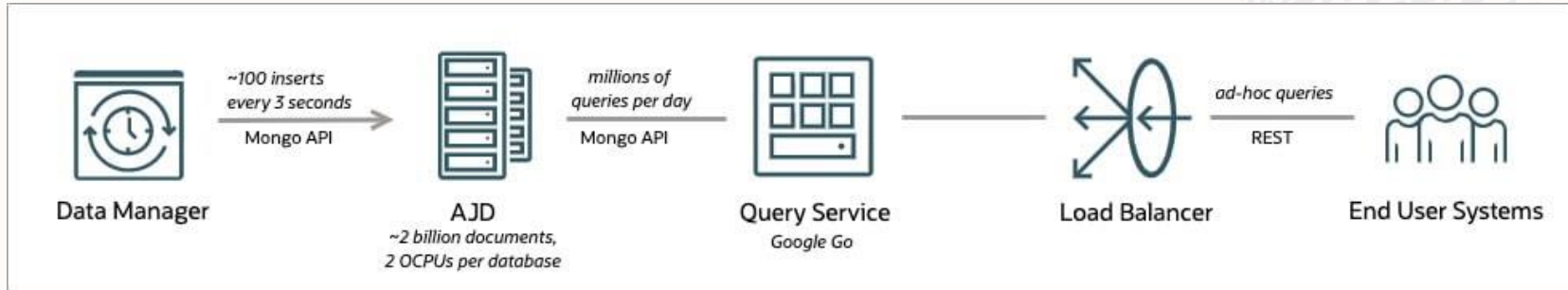
- Single converged database engine, providing **both SQL and MongoDB API access**

Customer benefits

- Reduced complexity
- Easier management
- Focus on functionality, not operational processes
- Enhanced user experience

* Case study [here](#)

MongoDB API - Cloud Success Stories



InfStones

- Using **MongoDB API drivers with Google Go**, no knowledge about the database
- Requirements: zero database administration with low-latency, scalable, development
- Technical [blog post](#) about their usage of AJD and MongoDB API for their multi-terabyte system

Oracle Backup Service

- Using **MongoDB API drivers with Java Micronaut**, no knowledge about the database
- Central internal Oracle on ATP service to backup all corporate laptops world-wide
- Limited rollout, full production expected in appr. 3 months

MongoDB API

European industry company

Migration in progress

MongoDB databases

Complex multi-database MongoDB ecosystem

- Hundreds of databases and collections
- Complex, error-prone, and slow time-to-prod

New architecture using Oracle Exadata and MongoDB API

- Single converged database engine, providing **both SQL and MongoDB API access**

POC completed successfully

Oracle Exadata with MongoDB API

Customer benefits

- Reduced complexity
- Easier management
- Focus on functionality, not operational processes
- Enhanced user experience

MongoDB API

European industry company

Migration in progress

AtlasDB

High costs of maintenance and subscription

- Large, multiterabyte volume of the data
- Use AtlasDB-specific extentions
 - Spatial and text search
- Complex, error-prone, and slow time-to-prod

New architecture using Oracle Exadata and MongoDB API

- Single converged database engine, providing **both SQL and MongoDB API access**

Oracle Exadata with MongoDB API

Customer benefits

- Reduced complexity
- Easier management
- Focus on functionality, not operational processes
- Enhanced user experience

Q&A
